

Writeup of Multi-SSD Model Ideas/Thoughts

A simplification of how the collections primitives all work

Focusing just on streaming type workloads (so like pattern matching, linefit, big int addition and the bucketing/filtering steps of samplesort and convex hull but not fft, or bellman ford) we can roughly think of the algorithm as having chunks coming in and chunks coming out.

More specifically the unordered file reader is maintaining some queue of sqes and as io_uring fulfills these asynchronously via DMA and brings in cqes, the file reader is concurrently handing these buffers off to compute threads to do some work on them, and we might have modified buffers coming back and creating write requests at the same time depending on the workload.

So then a simplification of what's going on in the algorithm is that at any point in time we have reads and writes that use up our limited memory bandwidth, and we have compute threads that turn the reads into writes.

This is meaningfully different than the usually assumed model of external algorithms, for example in pebble games we think of the CPU time pebbling step as instant and are only concerned with asking how many transfers we need to do between RAM and Disk, with block transfers being synchronous. Here IOs aren't privileged over CPU work and both happen at the same time, the tension is in how the two interact up to constants.

What makes an algorithm compute bound vs IO bound is how the rate at which the chunks come in compares to the rate at which they're consumed. In particular we can think of it as a queueing/throughput problem rather than just a question of IO work/depth

The basic model idea

Suppose that the machine, on some algorithm, consumes chunks at some fixed rate c (so this is like the chunks per second, absorbing details about algorithm/CPU speed), we're then interested in whether or not the file reader provides chunks at rate at least c .

Supposing that we're doing just one reading pass over the data (like reduce), we'd then expect that we need our ram bandwidth to be $2c$ in order to be competitive with an ideal implementation that's not memory bound, with the factor of two accounting for the fact that we're sharing bandwidth between reads from SSDs into RAM, and reads from RAM into cache.

If we're doing something like map then we will have reads and writes concurrently, so at any point we have reads and writes going out for each chunk in the input, so we would want bandwidth like $4c$ to make sure that our CPU is always being supplied with chunks

Why constants matter

What we'd ultimately like to characterize is whether or not we're CPU bound which is a good proxy for in-memory competitiveness so it's not just IO span that determines whether or not we'll be efficient. In fact most of the streaming type workloads are all $O(n)$ CPU work and $O(n/B)$ IO work, $O(1)$ (or maybe $O(B)$ depending on whether we count a block of work as a single unit of time) CPU span and $O(1)$ IO span.

So then there's something more detailed than just the IO span that's predictive, it's our bandwidth and IO demand relative to the CPU speed. For example if you were to compare just running map that increments each element versus a map that does something very expensive over each element, then the latter has a much lower value of c so you don't need as much bandwidth to be competitive despite the fact that both have the same IO span.

This also means that if you had an algorithm that, for each chunk, did work that was increasing in n then you'd guarantee competitiveness for a sufficiently large n as c would approach zero as n increases. I can't think of an example though, I mean sample sort kind of does this because the binary search step takes longer as the input size increases but the bucketing step can't really be modeled in this way

Delaying

Originally my mental picture was like suppose we have r reads and w writes going out any point in time, then our needed bandwidth is $2(r + w) \cdot c$, and so delaying is good because we're cutting down r, w . As Alex pointed out actually you would never have $r, w > 1$ as a workload that does multiple reads/writes per element would do each sequentially (for example if you do $\text{map} \rightarrow \text{map} \rightarrow \text{map}$), the three maps are entirely sequential and so at any point in time you're just doing a single map)

So then the benefit of delaying in this model is that it gives you a different value of c although I think maybe this is an idealistic view; thinking about a workload like we want to map an integer x and increment it 100 times, we can consider the relative speeds of doing this as 100 maps from x to $x + 1$ versus a single map that increments each x 100 times.

Supposing that the former has some rate of consuming chunks c , we would want bandwidth like $4c$ to make sure the CPU is always going over all 100 maps, whereas if we assume that the latter has a rate $c/100$ (so we're doing more work per element) then we need bandwidth $c/25$. So the benefit of delaying is that we need less bandwidth to keep the CPU happy, because the CPU doesn't consume elements as quickly

Things that are still unclear

What I think is sort of weird/needs to be thought about more is that you can also think of delaying as being beneficial because you're cutting down the constant in the IO span, like in the previous example the IO span is $O(1)$ in either case but you're changing the constant by delaying. These seem related most of the time; by cutting down the IO span you're compressing the IOs into passes that do more work per element which lessens the bandwidth required to saturate CPU

Maybe there's a simpler way to think of this though, and also a way to incorporate the random read type algorithms. The parallel disk model isn't quite what we want because it assumes synchronous IOs where CPU time is negligible. I also want to look back into pebble games, maybe there's some way to model these workloads as trees that could give a nice abstraction as the above still feels messy (which is maybe why all existing models abstract away CPU time it could just be inherently difficult to track both resources and their interaction)